

Implementing User Interfaces with HTML

1 Introduction

This assignment describes how to install the development environment for developing Web applications this semester. We will add new content every week, pushing the code to a **GitHub** source repository, and then deploying the content to a remote server hosted on **Netlify** and **Render.com**

The **World Wide Web** is a collection of dynamic and static documents hosted in a global network called the **Internet**. Servers connected to the Internet implement a **Client-Server** architecture where **clients** called **browsers**, request for **Web Pages** formatted in **HTML (HyperText Markup Language)** from **server** computers. Client browsers and servers talk to one another using **HTTP (HyperText Transport Protocol)**, a network communication protocol specifically designed for transporting **HTML** content meant to be parsed and rendered in Web browsers.

Web pages consist of plain text documents formatted with **HTML**, a dialect of **XML (eXtensible Markup Language)**. **HTML** is a computer language used to format the content displayed in Web pages such as foreground and background color, white spaces, text alignment, font, lists, tables, and forms. **HTML** documents are parsed by browsers which create an in-memory object representation called the **DOM (Document Object Model)**. The **DOM** consists of a hierarchical data structure representing the **HTML** document where each node in the hierarchy is configured to render the content in a particular style. In this assignment we will learn how to use HTML to format Web pages and create user interfaces.

JavaScript is a programming language that is most famous for being able to run within browsers and programmatically manipulating the **DOM**, and hence controlling what the browser renders on the screen. In this course we will be learning how to use **JavaScript** to implement user interfaces within browsers. **JavaScript** can also run outside of browsers to implement general purpose program. In later chapters we will use it specifically to implement **HTTP** server applications.

React.js is a **JavaScript** library designed to simplify creating application user interface meant to be viewed in browsers and interact with **HTTP** resources such as **HTML** content, **CSS** styling, images, video, etc. **React.js** applications consist of a collection of **JavaScript** functions called **components**, that implement an algorithm to calculate the user interface rendered on a browser. **React.js** applications are basically functions that transform user inputs, data structures, and server resources, into an equivalent user interface, visually representing the current state of an application. We refer to these applications as **Web applications** to highlight that these applications run on and use Web related technologies such as **HTML**, **CSS**, **JavaScript**, interacting with resources hosted by **HTTP servers**.

In this assignment we will create a **React.js** application and get started learning about **HTML** generated by **JavaScript** functions that implement **React.js components**. We will learn how to use various **HTML** elements to render user interface content such as headings, paragraphs, lists, tables, and form elements. The **Lab** section contains several exercises that introduce basic **HTML** elements and concepts giving you the opportunity to learn and practice **HTML** skills. The exercises in the **Labs** section provide detailed instructions to successfully accomplish the exercise, geared towards introducing concepts and practicing new skills.

The **Kanbas** section contains exercises where students are asked to apply the skills they learned in the **Labs** section, but to implement an actual **Web application**. The exercises provide some sample code and requirements, but deliberately leave out specific steps to complete the exercises. Students are expected to experiment and discover how to implement the requirements using the skills they learned in the **Labs**. The **Kanbas Web application** we are building consists of an **Online Learning Management System (OLMS)** inspired by a popular **OLMS** called **Canvas**, which students will recognize from the screen shots provided in the exercises. This assignment will focus on using plain **HTML** to implement rough content and layout of various **Kanbas** screens which at first won't look like the target screen shots. In later assignments

we will continue working on the **Kanbas** application by using **Cascade Style Sheets (CSS)** to style the Web pages so they look more like the screeb shots.

2 Labs

This section walks through several exercises to become familiar with **HTML**. First a development environment is setup enabling practicing several **HTML** exercises. Copy the examples into an IDE as instructed and confirm that the code renders and behaves as intended. After practicing with the exercises, apply the skills to implement **Kanbas**, a Web application inspired by a popular online learning management system.

2.1 Web Servers

In this course will be creating Web servers using the **JavaScript** programming language running on local and remote computers. To run **JavaScript** we will need **Node.js**, a popular **JavaScript runtime environment**. Navigate to <https://nodejs.org/> and download **Node.js** for the relevant operating system. Download the recommended version and install it on the local computer.

2.2 React.js

When someone visits a Web page on their browser, they are viewing content formatted using the **HyperText Markup Language (HTML)**. Browsers parse the **HTML** and create an in memory data structure called **Document Object Model (DOM)**. The browser uses the **DOM** to render the content on the screen meant to be viewed and interact with users, hence the name **user interfaces**. User interfaces are usually the first layer in a set of layers that implement an application, and is often referred to as the **front end**. As users and developers demanded more features and capabilities from Web pages, the industry evolved techniques, tools and frameworks to deal with the growing complexity. Today's front end frameworks implement advanced software engineering techniques and design patterns such as: object oriented programming, component technology, singletons, services, model view controllers (**MVC**). There are many front end frameworks to choose from and they all implement some or all of the techniques: Angular, Ember, Sail, Meteor, React.js, Vue.js, Dojo.

This course will use **React.js** since it has become one of the most popular front end **JavaScript** libraries in the industry. Using **npx**, a **Node.js** tool part of the **Node.js** installation, create a **React.js** project where we will be learning all about Web development. From the **home** directory (~), create a directory for this semester, and then another directory under that for this course. Below are examples of creating directories from a home directory of the file system.

On **macOS**, start the **Terminal** application. On a **Windows OS**, start the **console** application or **PowerShell**. On either OS, type the following to create the directory for the current year and term, e.g., **~/2049/winter/webdev**

```
$ cd ~ # navigate to your home directory in your file system
$ mkdir 2049 # creates a directory for the current year, e.g., year 2049
$ mkdir 2049/winter # creates a directory for the current term, e.g., winter inside 2049
$ mkdir 2049/winter/webdev # creates a directory called webdev inside winter/2049
$ cd 2049/winter/webdev # navigates to the directory webdev inside winter/2049
```

Feel free to choose other places in the file system making sure all directory and file names:

- are all lowercase
- do not have spaces in them
- are inside directories that also meet these criteria

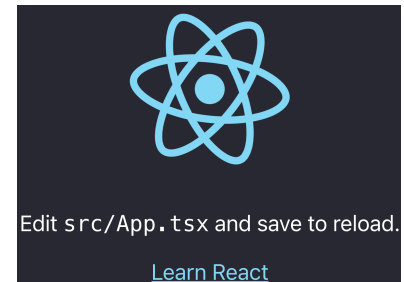
Still from the terminal or console, navigate to the directory created for this course and type the following to create a brand new **React.js** project called **kanbas-react-web-app** using the **create-react-app** Node.js module.

```
$ npx create-react-app kanbas-react-web-app --template typescript
```

A new directory called **kanbas-react-web-app** will be created and lots of libraries and code will be downloaded into the new directory. Wait for the process to complete and then navigate into the new directory and run the project.

```
$ cd kanbas-react-web-app
$ npm start
```

Using a favorite IDE, e.g., **Visual Code**, open the **kanbas-react-web-app** directory, and then the **src** directory. Do all the work under the **src** directory of the project. Starting from this assignment, create a separate **GitHub** branch for each assignment. Commit the work for this assignment into a branch called **a1**. Commit work in subsequent assignments 2, 3, 4, and 5 into branches **a2**, **a3**, **a4**, and **a5**. Once TAs grade the assignment, the branch can be merged into the **main** branch. We will configure **Netlify** so that each branch will deploy to a separate URL. Let's start learning how to use **HTML** to create a **User Interface**. Follow along in Chapters 1 and 2 of the **Full Stack Developer** book assigned for this course.



2.3 Creating the Labs Components

React.js is a **JavaScript** library that developers use to create dynamic Web user interfaces. Creating React.js user interfaces consists of writing functions, or **components**, that compute the content of the user interface in terms of **HTML** code snippets. Create the **Lab1** component shown below where we will be practicing **HTML**. Create the **Lab1** React.js component in the file **src/Labs/Lab1/index.tsx** as shown below.

src/Labs/Lab1/index.tsx

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
    </div>
  );
}
```

*// React.js function components can implement algorithms that compute
// an HTML code snippet and return the result of the computation.
// Other functions can aggregate various snippets from different
// components into a larger, single HTML content that browsers
// can then render on the screen. Here the component is just returning
// a hard coded, static HTML code snippet. Later assignments will
// introduce far more interesting algorithms to compute complex HTML*

The **Lab1** function above returns an **HTML** element **div** (division or divider) that contains an **h2** heading element with the text **Lab 1**. The **HTML** code snippet will render a heading of size 2 introducing the content of the first assignment. **React.js** components can be combined or aggregated into other components to create incrementally complex user interfaces. Import the **Lab1** component from a **Labs** component that will combine all the assignments worked throughout the semester. Create the **Labs** component in **src/Labs/index.tsx** as shown below.

src/Labs/index.tsx

```
import Lab1 from "../Lab1";
export default function Labs() {
  return (
    <div id="wd-labs">
      <h1>Labs</h1>
      <Lab1 />
    </div>
  );
}
```

*// Components can import other components to aggregate the code snippets
// of the components into larger, more complex HTML content. Here the
// Labs component imports the Lab1 component as the first of a
// set of assignments that will be implemented in later assignments.
// The Lab1 function is invoked with the HTML syntax <Lab1/>
// which is replaced by HTML the function returns implemented in the
// Lab1 function*

The **<Lab1 />** tag above uses **HTML** syntax to invoke the **Lab1** function and replace the tag with the value returned by **Lab1**. This **HTML** syntax allows seamlessly combining **JavaScript** and **HTML** syntax into a new syntax called **JSX** or **JavaScript XML**.

Generally all applications have an entry point. For instance the **main** function is a common entry point to applications written in the C and Java languages. These **main** functions typically delegate further computation to other functions, classes, and objects, creating a hierarchy of execution paths that flow from the root **main** function throughout various execution branches. Similarly the **App** component below is the top most, root component that is the entry point to the **React.js** application. Replace the default content in the **App** component with the code shown below to display the **Labs** component.

src/App.tsx

```
import React from "react";
import Labs from "../Labs";
export default function App() {
  return (
    <div>
      <Labs />
    </div>
  );
}
```

// The App component is the root component that imports other components
// to break up the HTML rendering task amongst a hierarchy of components

// For how we are just rendering the Labs component,
// which for now it's rendering just the Lab1 component

The computation of the user interface starts by invoking and evaluating **App()** which in turn calls **Labs()**, which in turn calls **Lab1()**. Each returns a snippet of **HTML** that combine to create the final, single **HTML** content that the browser displays.

The **App** component is invoked and rendered in the browser by the **index.tsx** file below using the **ReactDOM** library. The **render** function invokes the **App** function and injects the returned value into an element with the **ID** attribute equal to **root**, found in a static **HTML** file. Comment out, or delete the **index.css** import statement so we can see the raw, default style of Web pages. In later assignments we will learn how to style Web pages with **CSS**.

src/index.tsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import "../index.css";
import App from "./App";
import reportWebVitals from "./reportWebVitals";
const root = ReactDOM.createRoot(
  document.getElementById("root") as HTMLElement
);
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
reportWebVitals();
```

// comment out, or delete this line
// Leave the rest of this file alone

The **public** directory in the project contains the **index.html** file below which is the standard entry point of **Web pages**. When the project starts with **npm start**, a **Node.js** server starts running and makes the **index.html** Web page available to browsers at the <http://localhost:3000/index.html> URL (**Uniform Resource Locator**). Since **index.html** is the standard landing (entry) Web page, servers will typically allow abbreviating the **URL** to just <http://localhost:3000>, e.g., **index.html** is implied or assumed. The Web page contains a **div** element with the ID **root** which is initially empty at first. Don't edit the **index.html** file. The **ReactDOM**'s **render** function computes the **HTML** content from **App**, **Labs**, and **Lab1** component, and dynamically manipulates the **DOM** by inserting the content from **App** into the empty **div** element with the **root** ID. Start the application with **npm start** and confirm that the browser renders as shown below.

public/index.html

How the Browser Renders

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>React App</title>
  </head>
  <body>
```

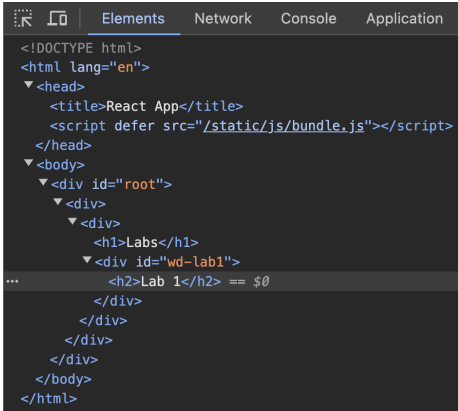
Labs

```
<div id="root"></div>
</body>
</html>
```

Lab 1

2.4 Browser Web Development Tools

The raw HTML of the Web page can be viewed by right clicking the Web page and selecting **View Page Source** and an abbreviated version is shown below on the left. Browsers also provide useful **Web development tools** or just **DevTools** accessible through **Ctrl+Shift+I** on Windows/Linux or **Cmd+Option+I** on macOS. The tools are also available by right clicking the browser window and selecting **Inspect**. Try it while viewing the **Labs** and confirm it looks as shown below on the right. The **Elements** tab in the **DevTools** displays the **DOM** as a hierarchical data structure representation of the Web page's source.

Page Source	DevTools
<pre><!DOCTYPE html> <html lang="en"> <head> <title>React App</title> <script src="/static/js/bundle.js" defer></script> </head> <body> <div id="root"></div> </body> </html></pre>	Labs Lab 1 

Notice that the page source on the left and the corresponding **DOM** on the right are fairly similar, up until you get to the content of the **<div id="root">**. In the **Page Source** the **div** element is empty, whereas in the **DOM**, the element has additional content including the **Labs** and **Lab 1** headings. The content in the **DOM** was dynamically added by the **script** element whose **src** attribute is referencing a file in **/static/js/bundle.js**. The **JavaScript** file is a **transpiled** version of all the collection of source files we have written so far: **App.tsx**, **Labs.tsx**, and **Lab1.tsx** which programmatically computes the **HTML** code to be dynamically injected into **<div id="root">**.

2.5 HTML Elements and Tags

Let's start learning about **HTML** by introducing some of the most common, but simple **elements** or **tags**. **HTML** is a special purpose dialect of **XML**, used for formatting plain text so that browsers know how to render the content in various styles. For instance the **HTML** snippet below styles the **Labs** text as a heading of size 1, and renders the text in large, bold, black letters in the browser.

```
<h1>Labs</h1>
```

When referring to the **<h1>** and **</h1>** in code, we use the term **tags** as in "**<h1>** is the **opening tag**" and "**</h1>** is the **closing tag**" and "the text **Labs** is wrapped with the **h1 tag**". When the "**<h1>Labs</h1>**" text is parsed by a browser, the tags are used to create an instance of a node object in a hierarchical data structure called the **DOM (Document Object Model)**. Since the concept of "heading of size 1" has two representations, one textual syntax, and another as a node in a hierarchical data structure, we often use **element** as an alternate, more generic term to refer to **h1** as an **element** instead of a **tag**. The distinction is so subtle that we will often use the **tag** and **element** terms interchangeably.

2.6 Heading Tags

Text documents are often broken up into several sections and subsections. Each section is usually prefaced with a short title or **heading** that attempts to summarize the topic of the section it precedes. For instance this paragraph is preceded by the heading **Heading Tags**. The font of the section headings are usually larger and bolder than the plain text and their subsection headings. This document uses headings to introduce topics such as HTML Documents, HTML Tags, Heading Tags, etc. HTML **heading tags** can be used to format plain text so that it renders in a browser as large headings. There are 6 heading tags for different sizes: **h1**, **h2**, **h3**, **h4**, **h5**, and **h6**. Tag **h1** is the largest heading and **h6** is the smallest heading.

To practice using the heading tags, let's create several headings and subheadings to introduce the topics we will cover in this assignment. In the **Lab1** component copy the HTML below and confirm that the browser refreshes with the new content as shown below on the right. Note how the text surrounded by the `<h2>` tag is larger and bolder than the text surrounded by the `<h3>` tag, and both are larger than the text that has no tags around it. The **src/Labs/Lab1/index.tsx** document will contain all the lab exercises for this assignment.

src/Labs/Lab1/index.tsx

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
      <h3>HTML Examples</h3>
      <div id="wd-h-tag">
        <h4>Heading Tags</h4>
        Text documents are often broken up into several sections and
        subsections. Each section is usually prefaced with a short
        title or heading that attempts to summarize the topic of the
        section it precedes. For instance this paragraph is preceded by
        the heading Heading Tags. The font of the section headings are
        usually larger and bolder than their subsection headings. This
        document uses headings to introduce topics such as HTML
        Documents, HTML Tags, Heading Tags, etc. HTML heading tags can
        be used to format plain text so that it renders in a browser as
        large headings. There are 6 heading tags for different sizes:
        h1, h2, h3, h4, h5, and h6. Tag h1 is the largest heading and
        h6 is the smallest heading.
      </div>
    </div>
  );
}
```

Labs

Lab 1

HTML Examples

Heading Tags

Text documents are often broken up into several sections and subsections. Each section is usually prefaced with a short title or heading that attempts to summarize the topic of the section it precedes. For instance this paragraph is preceded by the heading Heading Tags. The font of the section headings are usually larger and bolder than the plain text and their subsection headings. This document uses headings to introduce topics such as HTML Documents, HTML Tags, Heading Tags, etc. HTML heading tags can be used to format plain text so that it renders in a browser as large headings. There are 6 heading tags for different sizes: h1, h2, h3, h4, h5, and h6. Tag h1 is the largest heading and h6 is the smallest heading.

2.7 Paragraph Tag

Browsers ignore white spaces such as extra spaces, tabs and newlines. To add space between different paragraphs we can use the paragraph tag `<p>` to wrap the text and add vertical spacing. To practice using the paragraph tag, copy the code here on the right to the end of the **index.tsx**.

```
<div id="wd-p-tag">
  <h4>Paragraph Tag</h4>
  <p id="wd-p-1">
    This is a paragraph. We often separate a long set of sentences with
    vertical spaces to make the text easier to read. Browsers ignore vertical
    white spaces and render all the text as one single set of sentences. To
    force the browser to add vertical spacing, wrap the paragraphs you want to
    separate with the paragraph tag
  </p>
</div>
```

Below is another example of how the browser renders HTML text on the left column. Note how the browser ignores line breaks and other white space formatting such as tabs, new lines and extra spaces, and content just flows from left to right and then wraps when there's no more horizontal space. This rendering behavior is referred to as **inline**. Inline content flows from left to right horizontally the whole width of its parent container and then wraps vertically when there's no more horizontal space. Add the content below on the left and confirm that it renders as shown below on the right.

src/Labs/Lab1/index.tsx	How the browser renders
<pre>export default function Lab1() { return (<div id="wd-lab1"> <h2>Lab 1</h2> <h3>HTML Examples</h3> <div id="wd-h-tag"> <h4>Heading Tags</h4> ... </div> <div id="wd-p-tag"> <h4>Paragraph Tag</h4> <p id="wd-p-1"> ... </p> </div> </div>); }</pre> This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.	This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. This is the second paragraph. Even though we added a deliberate gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.

Apply the **paragraph tags** as shown below to let the browser know we want to keep the vertical spacing. Make sure to include the **id** attributes with the values shown. Confirm the paragraphs now render as shown below on the right. Note how the paragraphs are now spaced vertically from one another. Both the paragraph and heading tags add vertical space and we refer to this rendering behavior as **block**. By controlling the **inline** and **block** styles of laying out content, we can achieve all sorts of useful layouts.

src/Labs/Lab1/index.tsx	How the browser renders
<pre><div id="wd-p-tag"> <h4>Paragraph Tag</h4> <p id="wd-p-1"> ... </p> <p id="wd-p-2"> This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. </p> <p id="wd-p-3"> This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. </p> <p id="wd-p-4"> This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps. </p> </div></pre>	This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one. This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right. This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.

2.8 List Tags

2.8.1 Ordered List Elements

List elements are used to create lists of related items. There are two types of lists: **ordered** and **unordered**. Ordered list elements are useful for listing items in a particular order. Here's a list of steps to make pancakes. Add the content on the left after the paragraph exercise. Confirm the browser renders as shown on the right.

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
      <h3>HTML Examples</h3>
      <div id="wd-h-tag"> ... </div>
      <div id="wd-p-tag"> ... </div>
      <div id="wd-lists">
        <h4>List Tags</h4>
        <h5>Ordered List Tag</h5>
        How to make pancakes:
        1. Mix dry ingredients.
        2. Add wet ingredients.
        3. Stir to combine.
        4. Heat a skillet or griddle.
        5. Pour batter onto the skillet.
        6. Cook until bubbly on top.
        7. Flip and cook the other side.
        8. Serve and enjoy!
      </div>
    </div>
  );
}
```

Paragraph Tag

This is a paragraph. We often separate a long set of sentences with vertical spaces to make the text easier to read. Browsers ignore vertical white spaces and render all the text as one single set of sentences. To force the browser to

This is the first paragraph. The paragraph tag is used to format vertical gaps between long pieces of text like this one.

This is the second paragraph. Even though there is a deliberate white gap between the paragraph above and this paragraph, by default browsers render them as one contiguous piece of text as shown here on the right.

This is the third paragraph. Wrap each paragraph with the paragraph tag to tell browsers to render the gaps.

List Tags**Ordered List Tag**

How to make pancakes: 1. Mix dry ingredients. 2. Add wet ingredients. 3. Stir to combine. 4. Heat a skillet or griddle. 5. Pour batter onto the skillet. 6. Cook until bubbly on top. 7. Flip and cook the other side. 8. Serve and enjoy!

Note that in the HTML text on the left we explicitly wrote the numbers 1., 2., etc., but the formatting is lost when the browser renders it on the right. Instead of rendering a list of items, each in its own line, they are instead all rendered on the same line. Use the ordered list tag to achieve the desired format. The ordered list tag actually consists of a pair of tags

- `<ol>` declares the beginning of the list
- `<li>` declares an item in the list

Here's the same example from earlier, but now applying the ordered list tags to achieve the intended formatting.

```
<div id="wd-lists">
  <h4>List Tags</h4>
  <h5>Ordered List Tag</h5>
  How to make pancakes:
  <ol id="wd-pancakes">
    <li>Mix dry ingredients.</li>
    <li>Add wet ingredients.</li>
    <li>Stir to combine.</li>
    <li>Heat a skillet or griddle.</li>
    <li>Pour batter onto the skillet.</li>
    <li>Cook until bubbly on top.</li>
    <li>Flip and cook the other side.</li>
    <li>Serve and enjoy!</li>
  </ol>
</div>
```

List Tags**Ordered List Tag**

How to make pancakes:

1. Mix dry ingredients.
2. Add wet ingredients.
3. Stir to combine.
4. Heat a skillet or griddle.
5. Pour batter onto the skillet.
6. Cook until bubbly on top.
7. Flip and cook the other side.
8. Serve and enjoy!

Add the `<ol>` and `<li>` elements as shown above and confirm it renders as shown on the right. Create a new ordered list describing favorite recipes. Include it in an ordered list whose ID is `"wd-your-favorite-recipe"` and it should have at least 3 steps to make.

```
<div id="wd-lists">
  <h4>List Tags</h4>
  <h5>Ordered List Tag</h5>
  How to make pancakes:
  <ol id="wd-pancakes"> ... </ol>
  My favorite recipe:
  <ol id="wd-your-favorite-recipe">
```

```
    { /* complete on your own */ }  
  </ol>  
</div>
```

2.8.2 Unordered List Elements

The unordered list element is similar to its ordered version with the difference that the items are not numbered and instead bullets decorate each line item. The unordered list tag is `<ul>`, but the list item tag is still `<li>` as shown below. Unordered lists are great for displaying a list of items in no particular order. Here's an example of an unordered list of my favorite books in no particular order. Add the example HTML code below after the end of the previous ordered list exercise and then confirm the browser renders as shown below. Add an unordered list that contains at least 3 favorite books.

src/Labs/Lab1/index.tsx	How the browser renders
<pre>My favorite recipe: <ol id="wd-my-favorite-recipe">{ /* complete on your own */ } <h5>Unordered List Tag</h5> My favorite books (in no particular order) <ul id="wd-my-books"> Dune Lord of the Rings Ender's Game Red Mars The Forever War Your favorite books (in no particular order) <ul id="wd-your-books"> { /* complete on your own */ } </pre>	Unordered List Tag My favorite books (in no particular order) • Dune • Lord of the Rings • Ender's Game • Red Mars • The Forever War Your favorite books (in no particular order)

2.9 Table Tags

HTML began as a tool for sharing research results between scientists. These documents often consisted of data points captured as a result of some experiment. Each data point might have several attributes associated. A common way to display or visualize these results was formatted in a data table with a row for each data point and a column for each attribute. The `<table>` tag allows formatting data into a table with rows and columns. For instance, consider capturing grade results for several quizzes someone might have taken over a semester. These might be captured using the following table.

Quiz	Topic	Date	Grade
Q1	HTML	2/3/21	85
Q2	CSS	2/10/21	90
Q3	JavaScript	2/17/21	95
Average			90

Several things to note:

1. The first row is formatted as headings for each column
2. There are 3 data points, one for each quiz, one in each row
3. Each data point has the same data types for each of the columns, e.g, **Quiz** and **Topic** is text, **Date** is type date, and **Grade** is of type number
4. The last row is formatted as a footer
5. The three first columns of the last row are merged into a single cell and unlike the 3 data rows

HTML **table** tag can contain additional tags to format the data as follows:

- **table** - declares the start of a table
- **tr** - declares the start of a row
- **td** - declares a table data cell
- **thead** - declares a row of headings
- **tbody** - declares the main data content rows of the table
- **tfoot** - declares a row as a footer
- **th** - declares a table cell as a heading

To practice using **table** tag, copy the HTML below to the end of **index.tsx**. The code implements the table shown earlier. Ignore the comments on the right.

src/Labs/Lab1/index.tsx

```
export default function Lab1() {
  return (
    <div id="wd-lab1">
      <h2>Lab 1</h2>
      <h3>HTML Examples</h3>
      <div id="wd-h-tag"> ... </div>
      <div id="wd-p-tag"> ... </div>
      <div id="wd-lists"> ... </div>
      <div id="wd-tables">
        <h4>Table Tag</h4>
        <table border={1} width="100%">
          <thead>
            <tr>
              <th>Quiz</th>
              <th>Topic</th>
              <th>Date</th>
              <th>Grade</th>
            </tr>
          </thead>
          <tbody>
            <tr>
              <td>Q1</td>
              <td>HTML</td>
              <td>2/3/21</td>
              <td>85</td>
            </tr>
            <tr>
              <td>Q2</td>
              <td>CSS</td>
              <td>2/10/21</td>
              <td>90</td>
            </tr>
            <tr>
              <td>Q3</td>
              <td>JavaScript</td>
              <td>2/17/21</td>
              <td>95</td>
            </tr>
          </tbody>
          <tfoot>
            <tr>
              <td colspan={3}>Average</td>
              <td>90</td>
            </tr>
          </tfoot>
        </table>
      </div>
    </div>
  );
}
```

<!-- declares the table, sets border and width -->
 <!-- declares the table heading section -->
 <!-- declares the headings row -->
 <!-- declares heading for first column -->
 <!-- declares heading for second column -->
 <!-- declares heading for third column -->
 <!-- declares heading for fourth column -->

 <!-- declares the table's main content -->
 <!-- declares the first row -->
 <!-- declares data for first row, first column -->
 <!-- declares data for first row, second column -->
 <!-- declares data for first row, third column -->
 <!-- declares data for first row, fourth column -->

 <!-- declares the second row -->
 <!-- declares data for second row, first column -->
 <!-- declares data for second row, second column -->
 <!-- declares data for second row, third column -->
 <!-- declares data for second row, fourth column -->

 <!-- declares the third row -->
 <!-- declares data for third row, first column -->
 <!-- declares data for third row, second column -->
 <!-- declares data for third row, third column -->
 <!-- declares data for third row, fourth column -->

 <!-- declares the table footer section -->
 <!-- declares the footer row -->
 <!-- declares data spans 3 columns -->
 <!-- declares data for fourth column -->

Content in a table cell can be aligned at the top of the cell **td** using the **valign** attribute set to **"top"**, e.g., **<td valign="top">**. Content can also be aligned horizontally in a table cell with the **align** attribute, e.g., **<td align="right">** aligns the content to the right. This course has 10 quizzes. Add the 7 quizzes that are missing in the table. Use quiz names such as **Q3**, **Q4**, through **Q10**. Make up different dates and scores.

2.10 Image Tag

Use the image tag to render pictures in HTML documents. The images can be anywhere on the internet, or a local image document in the local file system.

```

```

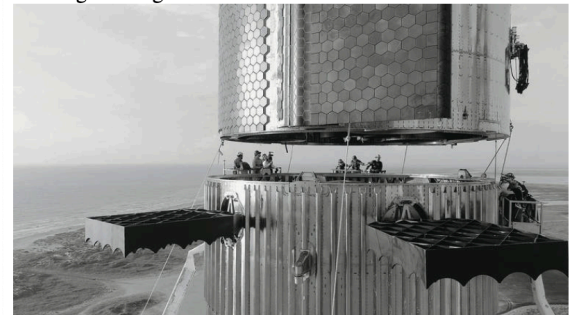
<!-- Use img tag to embed pictures in HTML documents. src attributes references image file either locally or remotely. width and height attributes configure the image size. If only width or height is provided, the other scales proportionally -->

To practice using the image tag, copy the code below to **index.tsx**. The first image tag embeds an image from a remote server. The second one assumes there's a local image file called **teslabot.jpg** in **public/images/teslabot.jpg**. Search for **Tesla Bot** on the internet, and download an image that looks similar to the one shown below. Name the image **teslabot.jpg**.

```
<div id="wd-images">
  <h4>Image tag</h4>
  Loading an image from the internet:
  <br />
  
  <br />
  Loading a local image:
  <br />
  
</div>
```

Image tag

Loading an image from the internet:



Loading a local image:



2.11 Form Tags

Form tags are useful for entering data. Let's take a look at the most common ones: **form**, **input**, **select**, **textarea**, **radio**, **checkbox**.

2.11.1 Text fields

Text fields are the most common form elements allowing entering a single line of text.

```
<input type="text"
        placeholder="hint"
        title="tooltip"
        value="COMEDY"/>
```

<!-- use input tag's text type to declare a single line input field text is default if type is left out. Use placeholder and title to give a hint of what information you're expecting. Optionally initialize the value of the field with value attribute-->

To practice using text fields, add the following code snippet at the end of **index.tsx**. It creates a set of input fields for entering some personal information. The **label** tags below associate descriptive text with each form element by setting a **label's htmlFor** attribute to the **id** attribute of the related form field.

```
<div id="wd-forms">
  <h4>Form Elements</h4>
  <form id="wd-text-fields">
    <h5>Text Fields</h5>
    <label htmlFor="wd-text-fields-username">Username:</label>
    <input id="wd-text-fields-username" placeholder="jdoe" /> <br />
    <label htmlFor="wd-text-fields-password">Password:</label>
    <input type="password" id="wd-text-fields-password" value="123@#$asd" />
    <br />
    <label htmlFor="wd-text-fields-first-name">First name:</label>
    <input type="text" id="wd-text-fields-first-name" title="John" /> <br />
    <label htmlFor="wd-text-fields-last-name">Last name:</label>
    <input type="text" id="wd-text-fields-last-name" placeholder="Doe"
      value="Wonderland" title="The last name" />
    { /* copy rest of form elements here */ }
  </form>
</div>
```

Form Elements

Text Fields

Username:

Password:

First name:

Last name:

2.11.2 Textareas

The **textarea** tag is useful for entering long form text such as someone's biography data, or a blog post.

```
<textarea cols="20" rows="25"
  placeholder="Biography"
  title="tooltip">Some text
</textarea>
```

<!-- use textarea tag for long form text configure its width and height with attributes cols and rows. Use placeholder and tooltip to give hints. Note default value is in tag's body -->

To practice using the **textarea** tag, add the following example to the end of **index.tsx**. It creates a **textarea** useful for entering a biography. Get a sample of the dummy text at <https://www.lipsum.com>.

```
<h5>Text boxes</h5>
<label>Biography:</label><br/>
<textarea id="wd-textarea" cols={30} rows={10}>Lorem ipsum dolor sit
amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis
nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit
esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat
cupidatat non proident, sunt in culpa qui officia deserunt mollit
anim id est laborum.</textarea>
```

Text boxes

Biography:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in

2.11.3 Buttons

Buttons allow invoking actions executed by the browser. To practice creating buttons, copy the code below at the end of **index.tsx**. Confirm that clicking the button displays an alert box with an optimistic message.

```
<h5 id="wd-buttons">Buttons</h5>
<button id="wd-all-good" onClick={() => alert("Life is Good!")} type="button">
  Hello World!
</button>
```

2.11.4 File upload button

Use the file type for the input tag to choose a file for upload. We won't be able to upload files just yet until later in the course, but for now let's practice adding a file upload tag as shown below. Clicking the button pops up a file chooser to navigate to the file to upload. To practice using the file selector, copy the code below to the end of **index.tsx**.

```
<h5>File upload</h5>
<input id="wd-upload" type="file"/>
```

2.11.5 Radio buttons

Radio buttons allow selecting a single choice from multiple alternative options

```
<input type="radio" name="NAME1"
value="OPTION1"/>
<input type="radio" name="NAME1"
value="OPTION2" checked/>
/* use the input tag's checkbox type to declare a checkbox */
/* use checked to set the checkbox's as initially checked */
```

To practice using radio buttons, add the following example at the end of **index.tsx**. The **label** tags allow associating a piece of text with an input field by setting its **htmlFor** attribute to the **id** of the corresponding input field. For instance, the **<label htmlFor="wd-radio-comedy">Comedy</label>** is associated with **<input id="wd-radio-comedy"/>** because the **label's** **htmlFor** attribute has the same value as the **id** attribute of the **input** field. Now, when clicking on the label, it's as if having clicked on the associated input field. In the case of a **checkbox**, the value toggles between **checked** or not. In the case of a **text** input field, the field gets focus. To make radio buttons **mutually exclusive**, group them by setting their **name** attributes to the same value. When a radio button is clicked in the same group, all the other radio buttons are unselected.

```
<h5 id="wd-radio-buttons">Radio buttons</h5>
<label>Favorite movie genre:</label><br />
<input type="radio" name="radio-genre" id="wd-radio-comedy"/>
<label htmlFor="wd-radio-comedy">Comedy</label><br />
<input type="radio" name="radio-genre" id="wd-radio-drama"/>
<label htmlFor="wd-radio-drama">Drama</label><br />
<input type="radio" name="radio-genre" id="wd-radio-scifi"/>
<label htmlFor="wd-radio-scifi">Science Fiction</label><br />
<input type="radio" name="radio-genre" id="wd-radio-fantasy"/>
<label htmlFor="wd-radio-fantasy">Fantasy</label>
```

Radio buttons

Favorite movie genre:

- ☐ Comedy
- ☐ Drama
- ☒ Science Fiction
- ☐ Fantasy

2.11.6 Checkboxes

Checkboxes allow selecting multiple choices

```
<input type="checkbox" name="NAME2"
value="OPTION1" checked/>
<input type="checkbox" name="NAME2"
value="OPTION2"/>
<input type="checkbox" name="NAME2"
value="OPTION3" checked/>
/* use the input tag's checkbox type to declare a checkbox */
```

To practice using checkboxes, add the following example to the end of **index.tsx**. It creates a set of checkbox buttons to select all the favorite movie genres, which there might be more than one.

```

<h5 id="wd-checkboxes">Checkboxes</h5>
<label>Favorite movie genre:</label><br/>

<input type="checkbox" name="check-genre" id="wd-chkbox-comedy"/>
<label htmlFor="wd-chkbox-comedy">Comedy</label><br/>

<input type="checkbox" name="check-genre" id="wd-chkbox-drama"/>
<label htmlFor="wd-chkbox-drama">Drama</label><br/>

<input type="checkbox" name="check-genre" id="wd-chkbox-scifi"/>
<label htmlFor="wd-chkbox-scifi">Science Fiction</label><br/>

<input type="checkbox" name="check-genre" id="wd-chkbox-fantasy"/>
<label htmlFor="wd-chkbox-fantasy">Fantasy</label>

```

Checkboxes

Favorite movie genre:

- ☒ Comedy
- ☐ Drama
- ☒ Science Fiction
- ☐ Fantasy

2.11.7 Dropdowns

Dropdowns are useful for selecting one or more options from a list of possible values. The default version displays a set of values from which one can choose a single value.

```

<select>
  <option value="VAL1">Value 1</option>
  <option value="VAL2" selected>Value 2</option>
  <option value="VAL3">Value 3</option>
</select>

```

*<!-- Wrap several **option** tags in a **select** tag. Optionally provide option's **value**, otherwise the option's text is the value of the select element. Optionally use **selected** attribute to select default. -->*

Adding the optional **multiple** attribute converts the dropdown into a list of options that can be selected.

```

<select multiple>
  <option value="VAL1" selected>Value 1</option>
  <option value="VAL2">Value 2</option>
  <option value="VAL3" selected>Value 3</option>
</select>

```

*<!-- Alternatively use attribute **multiple** to allow selecting more than one option. Use **ctrl+click** to select more than one option -->*

To practice using the **select** tag, add the following example to the end of **index.tsx**. It creates a dropdown and a list of options.

```

<h4 id="wd-dropdowns">Dropdowns</h4>

<h5>Select one</h5>
<label htmlFor="wd-select-one-genre"> Favorite movie genre: </label><br/>
<select id="wd-select-one-genre">
  <option value="COMEDY">Comedy</option>
  <option value="DRAMA">Drama</option>
  <option selected value="SCIFI">
    Science Fiction</option>
  <option value="FANTASY">Fantasy</option>
</select>

<h5>Select many</h5>
<label htmlFor="wd-select-many-genre"> Favorite movie genres: </label><br/>
<select id="wd-select-many-genre" multiple>
  <option selected value="COMEDY">Comedy</option>
  <option value="DRAMA">Drama</option>
  <option selected value="SCIFI">
    Science Fiction</option>
  <option value="FANTASY">Fantasy</option>
</select>

```

Dropdowns

Select one

Favorite movie genre:

Science Fiction ▼

Select many

Favorite movie genres:

Comedy
Drama
Science Fiction
Fantasy

2.11.8 Other HTML field types

The input tag's **type** attribute has several other possible values: **date**, **email**, **number**, and **range**. They can be used to enter text information with a specific format. To practice these other formats add the following example under the last input field worked on earlier, but inside the **form** tag. The fields should look as shown below on the right.

```
<h4>Other HTML field types</h4>

<label htmlFor="wd-text-fields-email"> Email: </label>
<input type="email"
  placeholder="jdoe@somewhere.com"
  id="wd-text-fields-email"/><br/>

<label htmlFor="wd-text-fields-salary-start"> Starting salary:
</label>
<input type="number"
  id="wd-text-fields-salary-start"
  placeholder="1000"
  value="100000"/><br/>

<label htmlFor="wd-text-fields-rating"> Rating: </label>
<input type="range" id="wd-text-fields-rating"
  placeholder="Doe"
  max="5"
  value="4"/><br/>

<label htmlFor="wd-text-fields-dob"> Date of birth: </label>
<input type="date"
  id="wd-text-fields-dob"
  value="2000-01-21"/><br/>
```

Other HTML field types

Email:

Starting salary:

Rating:

Date of birth:

2.12 Anchor Tag

The anchor tag allows navigating to other websites or other pages within the same website.

```
<a href="aa.com">
American Airlines</a>
```

```
<!-- Use the href attribute to refer to the location of the website or other
page in the same website. Click on the body text to navigate -->
```

To practice using anchor tags, add the following example at the end of **index.tsx**. It creates two hyperlinks. One navigates to **lipsum.com**, a website that contains dummy text. Create another link to the code repository on GitHub. Click on the links and confirm the navigation works. Give the anchor an ID value of **wd-github**.

```
<h4>Anchor tag</h4>
Please
<a id="wd-lipsum" href="https://www.lipsum.com">click here</a>
to get dummy text<br/>
```

Anchor tag

Please [click here](https://www.lipsum.com) to get dummy text

2.13 Implementing Navigation in Single Page Applications

To implement navigation, install the **React Router** library from the command line as shown below. Run the command from the root of the project.

```
npm install react-router
npm install react-router-dom
```

The **React Router** library can be used to implement navigation in all kinds of devices including Web applications, mobile, and desktop. To implement navigation in Web application, also install the **React Router DOM** library as shown above. Once

the libraries are installed, use the **HashRouter** to implement navigation as shown below. The **HashRouter** tag sets up the base mechanism to navigate between multiple screens.

src/App.tsx

```
import Labs from "./Labs";
import { HashRouter } from "react-router-dom";
export default function App() {
  return (
    <HashRouter>
      <div>
        <Labs />
      </div>
    </HashRouter>
  );
}
```

Within a **Routes** element we can declare several **Routes** that will display an **element** only if the **URL** matches the pattern in the **path** attribute. The example below declares a **Route** that will display the **Labs** component only if the **URL** in the browser matches the **/Labs/*** pattern declared in the **path** attribute. Confirm that navigating to <http://localhost:3000> no longer displays the **Labs** and that they only display when navigating to <http://localhost:3000/#/Labs>.

src/App.tsx

```
import Labs from "./Labs";
import { HashRouter, Route, Routes } from "react-router-dom";
export default function App() {
  return (
    <HashRouter>
      <div>
        <Routes>
          <Route path="/Labs/*" element={<Labs />} />
        </Routes>
      </div>
    </HashRouter>
  );
}
```

We can make any component the default screen by forcing navigation with the **Navigate** component. The example below navigates to the **Labs** if the **URL** matches the pattern **/** declared in the **path** attribute. This effectively navigates to the **Labs** component by default. Confirm that navigating to <http://localhost:3000> displays **Labs**.

src/App.tsx

```
import Labs from "./Labs";
import { HashRouter, Route, Routes, Navigate } from "react-router-dom";
export default function App() {
  return (
    <HashRouter>
      <div>
        <Routes>
          <Route path="/" element={<Navigate to="Labs" />} />
          <Route path="/Labs/*" element={<Labs />} />
        </Routes>
      </div>
    </HashRouter>
  );
}
export default App;
```

As we work through the various assignments in this course, we're going to work on various labs discussing different topics. To practice some more navigation, let's create placeholders for upcoming labs and implement navigation to each of the labs. Create the following **Lab2** and **Lab3** components as shown below

src/Labs/Lab2/index.tsx

```
export default function Lab2() {
  return (
    <div>
      <h2>Lab 2</h2>
    </div>
  );
}
```

src/Labs/Lab3/index.tsx

```
export default function Lab3() {
  return (
    <div>
      <h2>Lab 3</h2>
    </div>
  );
}
```

In the **Labs** component, add the following **Routes** to navigate to **Labs**, **Lab1**, **Lab2**, and **Lab3**. Create a table of content (**TOC**) component that implements links we can click to navigate to the each lab. Confirm that the **Labs** component displays as the default landing screen, and clicking on each of the links navigates to the different labs.

src/Labs/index.tsx

```
import Lab1 from "../Lab1";
import { Route, Routes, Navigate } from "react-router";
import TOC from "../TOC";
import Lab2 from "../Lab2";
import Lab3 from "../Lab3";
export default function Labs() {
  return (
    <div>
      <h1>Labs</h1>
      <TOC />
      <Routes>
        <Route path="/" element={<Navigate to="Lab1" /> } />
        <Route path="Lab1" element={<Lab1 /> } />
        <Route path="Lab2" element={<Lab2 /> } />
        <Route path="Lab3" element={<Lab3 /> } />
      </Routes>
    </div>
  );
}
```

src/Labs/TOC.tsx

```
export default function TOC() {
  return (
    <ul>
      <li><Link to="/Labs">Labs</Link></li>
      <li><Link to="/Labs/Lab1">Lab 1</Link></li>
      <li><Link to="/Labs/Lab2">Lab 2</Link></li>
      <li><Link to="/Labs/Lab3">Lab 3</Link></li>
    </ul>
  );
}
```

Labs

- [Labs](#)
- [Lab 1](#)
- [Lab 2](#)
- [Lab 3](#)

3 Implementing the Kanbas User Interface

After practicing various aspects of **HTML**, use those skills to build **Kanbas**, a Web site based on a popular **Online Learning Management System (OLMS)**. The next several assignments will describe implementing several screens providing opportunities to practice the skills discussed in the lectures. This assignment will focus on implementing some simple, prototype versions of the more common screens improved on in later assignments. Create a new directory **src/Kanbas** and do all work in there.

3.1 Implementing the Kanbas Landing Page

Let's start by creating a **Kanbas** component that will be the entry point to the **Kanbas** user interface. Create the component in **src/Kanbas/index.tsx** as shown below.

src/Kanbas/index.tsx

```
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <h1>Kanbas</h1>
    </div>
  );
}
```

Map the new component to a new route to navigate to URL <http://localhost:3000/#/Kanbas> by default. In the table of component (**TOC.tsx**), add a new link to navigate to the **Kanbas** component as a separate screen. Confirm one can navigate to **Kanbas**.

src/App.tsx

```
import Labs from "../Labs";
import Kanbas from "../Kanbas";
export default function App() {
  return (
    <HashRouter>
      <div>
        <Routes>
          <Route path="/" element={<Navigate to="Kanbas"/>}/>
          <Route path="/Labs/*" element={<Labs />}/>
          <Route path="/Kanbas/*" element={<Kanbas />}/>
        </Routes>
      </div>
    </HashRouter>
  );
}
```

src/Labs/TOC.tsx

```
export default function TOC() {
  return (
    <ul>
      <li><Link to="/Labs">Labs</Link></li>
      <li><Link to="/Labs/Lab1">Lab 1</Link></li>
      <li><Link to="/Labs/Lab2">Lab 2</Link></li>
      <li><Link to="/Labs/Lab3">Lab 3</Link></li>
      <li><Link to="/Kanbas">Kanbas</Link></li>
    </ul>
  );
}
```

3.2 Implementing the Account Screens

The **Account Screens** provide users access to their personal information and all related data such as courses they are registered for and courses they might be teaching. Users use the **Sign Up** screen to register with the application. They can then use the **Sign In** screen to identify themselves and access their **Profile** screen to view and edit personal information.

3.2.1 Implementing the Sign In Screen

Signing into an application usually consists of providing credentials such as a **username** and **password**. The following component below declares input fields where users can enter their **username** and **password**.

src/Kanbas/Account/Signin.tsx

```
import { Link } from "react-router-dom";
export default function Signin() {
  return (
    <div id="wd-signin-screen">
      <h3>Sign in</h3>
      <input className="wd-username" placeholder="username" /> <br />
      <input className="wd-password" placeholder="password" type="password" /> <br />
      <Link id="wd-signin-btn" to="/Kanbas/Account/Profile">Sign in</Link> <br />
      <Link id="wd-signup-link" to="/Kanbas/Account/Signup">Sign up</Link>
    </div>
  );
}
```

Sign in

Sign in

[Sign up](#)

Kanbas

Create an **Account** screen and import the new **Signin** screen.

src/Kanbas/Account/index.tsx

```
import Signin from "../Signin";
export default function Account() {
  return (
    <div id="wd-account-screen">
      <h2>Account</h2>
      <Signin />
    </div>
  );
}
```

Account

Sign in

[Sign in](#)

[Sign up](#)

Import the new **Account** screen in the **Kanbas** screen and confirm the new **Signin** screen renders in the **Account** screen.

src/Kanbas/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
import Account from "../Account";
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <h1>Kanbas</h1>
      <Account />
    </div>
  );
}
```

3.2.2 Implementing the Profile Screen

Implement the **Profile** screen as shown below and include it in the **Account** screen below the **Signin** screen. Confirm that the **Kanbas** screen shows the **Account** screen with both the **Signin** and **Profile** screens.

src/Kanbas/Account/Profile.tsx

```
import { Link } from "react-router-dom";
export default function Profile() {
  return (
    <div id="wd-profile-screen">
      <h3>Profile</h3>
      <input defaultValue="alice" placeholder="username" class="wd-username" /><br/>
      <input defaultValue="123" placeholder="password" type="password" class="wd-password" /><br/>
      <input defaultValue="Alice" placeholder="First Name" id="wd-firstname" /><br/>
      <input defaultValue="Wonderland" placeholder="Last Name" id="wd-lastname" /><br/>
      <input defaultValue="2000-01-01" type="date" id="wd-dob" /><br/>
      <input defaultValue="alice@wonderland" type="email" id="wd-email" /><br/>
      <select defaultValue="FACULTY" id="wd-role">
        <option value="USER">User</option>
        <option value="ADMIN">Admin</option>
        <option value="FACULTY">Faculty</option>
        <option value="STUDENT">Student</option>
      </select><br/>
      <Link to="/Kanbas/Account/Signin" >Sign out</Link>
    </div>
  );
}
```

Profile

alice
password
Alice
Wonderland
01/01/2000
alice@wonderland.com
User
Sign out

3.2.3 Implementing the Sign Up Screen

Implement a **Signup** screen component as shown below. Import the **Signup** screen in the **Account** screen and confirm that the **Kanbas** screen displays the **Account** screen that renders the **Signin**, **Profile**, and **Signup** screens.

src/Kanbas/Account/Signup.tsx

```
import React from "react";
import { Link } from "react-router-dom";
export default function Signup() {
  return (
    <div id="wd-signup-screen">
      <h3>Sign up</h3>
      <input placeholder="username" class="wd-username" /><br/>
      <input placeholder="password" type="password" class="wd-password" /><br/>
      <input placeholder="verify password" type="password" class="wd-password-verify" /><br/>
      <Link to="/Kanbas/Account/Profile" >Sign up</Link><br />
      <Link to="/Kanbas/Account/Signin" >Sign in</Link>
    </div>
  );
}
```

Sign up

username
password
verify password
Sign up
Sign in

3.2.4 Account Screen Navigation

The **Account** screen navigation currently shows all three screens **Signin**, **Profile**, and **Signup**. Add routes to the **Account** and **Kanbas** screens so that these screens only show when users navigate to them. In the **Account** screen, add the following routes so that the **Signin** screen is the default.

src/Kanbas/Account/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
...
export default function Account() {
  return (
    <div id="wd-account-screen">
      <h2>Account</h2>
      <Routes>
        <Route path="/" element={<Navigate to="/Kanbas/Account/Signin" />} />
        <Route path="/Signin" element={<Signin />} />
        <Route path="/Profile" element={<Profile />} />
        <Route path="/Signup" element={<Signup />} />
      </Routes>
    </div>
  );
}
```

In the **Kanbas** screen, add the following routes so that the **Account** screen is the default. Confirm that the **Kanbas** screen shows the **Signin** screen by default.

src/Kanbas/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
import Account from "../Account";
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <h1>Kanbas</h1>
      <Routes>
        <Route path="/" element={<Navigate to="Account" />} />
        <Route path="/Account/*" element={<Account />} />
      </Routes>
    </div>
  );
}
```

Also confirm that clicking the **Sign in** button navigates to the **Profile** screen and clicking **Sign Out** in the **Profile** screen navigates back to the **Sign in** screen. Further confirm that clicking the **Sign Up** link in the **Sign in** screen navigates to the **Sign up** screen and that you can navigate back to the **Sign in** screen from the **Sign up** screen.

3.2.5 Account Navigation Sidebar

Create a **Account Navigation** component to provide navigation links to screens **Sign in**, **Sign up**, and **Profile** as shown below.

src/Kanbas/Account/Navigation.tsx

```
import { Link } from "react-router-dom";
export default function AccountNavigation() {
  return (
    <div id="wd-account-navigation">
      <Link to={`/Kanbas/Account/Signin`} > Signin </Link> <br/>
      <Link to={`/Kanbas/Account/Signup`} > Signup </Link> <br/>
      <Link to={`/Kanbas/Account/Profile`} > Profile </Link> <br/>
    </div>
  );
}
```

In the **Account** screen add a **table** to layout the **AccountNavigation** component on a column on the left and the routes on the right side column as shown below. Confirm that clicking the links in the **Account Navigation** sidebar actually navigates to the screens.

src/Kanbas/Account/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
import AccountNavigation from "../Navigation";
import Profile from "../Profile";
import Signin from "../Signin";
import Signup from "../Signup";
export default function Account() {
  return (
    <div id="wd-account-screen">
      <table>
        <tr>
          <td valign="top">
            <AccountNavigation />
          </td>
          <td valign="top">
            <Routes>
              <Route path="/"
                element={<Navigate to="/Kanbas/Account/Signin" />} />
              <Route path="/Signin" element={<Signin />} />
              <Route path="/Profile" element={<Profile />} />
              <Route path="/Signup" element={<Signup />} />
            </Routes>
          </td>
        </tr>
      </table>
    </div>
  );
}
```

Kanbas

[Signin](#)
[Signup](#)
[Profile](#)

Sign in

[Sign in](#)
[Sign up](#)

3.3 Implementing the Dashboard Screen

The **Kanbas Dashboard** lists courses as illustrated here on the right, giving access to courses a user is enrolled in as a student and courses a faculty is teaching. Clicking on a course navigates to that specific course. In a new **index.tsx** file under **src/Kanbas/Dashboard**, create the **Kanbas Dashboard** displaying at least 7 courses. Use the **Dashboard.tsx** file shown below as an example. Feel free to come up with additional courses. Download images for each course and save them to **public/images**. [The code below uses an image of the React.js logo](#).

src/Kanbas/Dashboard.tsx

```
import { Link } from "react-router-dom";
export default function Dashboard() {
  return (
    <div id="wd-dashboard">
      <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
      <h2 id="wd-dashboard-published">Published Courses (12)</h2> <hr />
      <div id="wd-dashboard-courses">
        <div className="wd-dashboard-course">
          <Link className="wd-dashboard-course-link"
            to="/Kanbas/Courses/1234/Home">
            
            <div>
              <h5>
                CS1234 React JS
              </h5>
            </div>
          </Link>
        </div>
        <div>
          <h5>
            CS4550 12631 Web Development...
            CS4550.12631.202410
            202410_1 Fall 2023 Semester Full Term
          </h5>
        </div>
        <div>
          <h5>
            CS4550 1753 Web Development...
            CS4550.1753.202410
            202410_1 Fall 2023 Semester Full Term
          </h5>
        </div>
        <div>
          <h5>
            CS5610 20040 Web Development...
            CS5610.20040.202310
            202310_2 Fall 2022 Semester Full Term ...
          </h5>
        </div>
        <div>
          <h5>
            CS5610 41752 Web Development...
            CS5610.41752.202340
            202340_2 Summer 1 2023 Semester Full...
          </h5>
        </div>
        <div>
          <h5>
            CS5610 11550 Web Development...
            CS5610.11550.202410
            202410_2 Fall 2023 Semester Full Term ...
          </h5>
        </div>
        <div>
          <h5>
            CS5610 17387 Web Development...
            CS5610.17387.202410
            202410_2 Fall 2023 Semester Full Term ...
          </h5>
        </div>
      </div>
    </div>
  );
}
```

Account

Dashboard

Courses

Calendar

Inbox

History

Studio

Commons

Help

Dashboard

Published Courses (24)

CS4550 12631 Web Development...
CS4550.12631.202410
202410_1 Fall 2023 Semester Full Term

CS4550 1753 Web Development...
CS4550.1753.202410
202410_1 Fall 2023 Semester Full Term

CS5610 20040 Web Development...
CS5610.20040.202310
202310_2 Fall 2022 Semester Full Term ...

CS5610 41752 Web Development...
CS5610.41752.202340
202340_2 Summer 1 2023 Semester Full...

CS5610 11550 Web Development...
CS5610.11550.202410
202410_2 Fall 2023 Semester Full Term ...

CS5610 17387 Web Development...
CS5610.17387.202410
202410_2 Fall 2023 Semester Full Term ...

/* Dashboard Title */
/* Published Courses */
/* Course 1 */

```

    </h5>
    <p className="wd-dashboard-course-title">
      Full Stack software developer
    </p>
    <button> Go </button>
  </div>
</Link>
</div>
<div className="wd-dashboard-course"> ... </div>
<div className="wd-dashboard-course"> ... </div>
</div>
</div>
);
}

```

```

{/* Course 2 */}
{/* Course 3 */}
{/* Add at Least 7 courses in total */}

```

The dashboard implementation does not have to look exactly like the screen shot shown for now. Later assignments will come back to this screen and apply **CSS** to style it as shown in the screen shot. Add a route in the **Kanbas** screen for the new **Kanbas Dashboard** screen

src/Kanbas/index.tsx

```

import { Routes, Route, Navigate } from "react-router";
import Account from "../Account";
import Dashboard from "../Dashboard";
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <h1>Kanbas</h1>
      <Routes>
        <Route path="/" element={<Navigate to="Account" />} />
        <Route path="/Account/*" element={<Account />} />
        <Route path="/Dashboard" element={<Dashboard />} />
      </Routes>
    </div>
  );
}

```

Dashboard

Published Courses (12)



[CS1234 React JS](#)

Full Stack software developer

[Go](#)

Change the **Sign in** link in the **Sign In** screen so that it navigates to the new **Dashboard** screen. Confirm that signing in navigates to the **Dashboard**.

src/Kanbas/Account/Signin.tsx

```

...
<div id="wd-signin-screen">
  <h3>Sign in</h3>
  <input className="wd-username" placeholder="username" /> <br />
  <input className="wd-password" placeholder="password" type="password" /> <br />
  <Link id="wd-signin-btn" to="/Kanbas/Dashboard">
    Sign in
  </Link>
  <br />
  <Link id="wd-signup-link" to="/Kanbas/Account/Signup">
    Sign up
  </Link>
</div>
...

```

3.3.1 Implement the Kanbas Navigation Sidebar

The first column of the **Kanbas Dashboard** contains the **Kanbas Navigation** sidebar, a list of hyperlinks (anchors) to navigate across multiple courses in the **Kanbas** application. Users can navigate to the **Dashboard** by clicking **Dashboard** on the **Kanbas Navigation** side bar. Implement the **Kanbas Navigation** sidebar in **src/Kanbas/Navigation.tsx**, shown here on the right. Use the code below as an example. Import all necessary components.

[Northeastern](#)
[Account](#)
[Dashboard](#)
[Courses](#)
[Calendar](#)
[Inbox](#)
[Labs](#)

src/Kanbas/Navigation.tsx

```
import { Link } from "react-router-dom";
export default function KanbasNavigation() {
  return (
    <div id="wd-kanbas-navigation">
      <a href="https://www.northeastern.edu/" id="wd-neu-link" target="_blank">Northeastern</a><br/>
      <Link to="/Kanbas/Account" id="wd-account-link">Account</Link><br/>
      <Link to="/Kanbas/Dashboard" id="wd-dashboard-link">Dashboard</Link><br/>
      <Link to="/Kanbas/Dashboard" id="wd-course-link">Courses</Link><br/>
      <Link to="/Kanbas/Calendar" id="wd-calendar-link">Calendar</Link><br/>
      <Link to="/Kanbas/Inbox" id="wd-inbox-link">Inbox</Link><br/>
      <Link to="/Labs" id="wd-labs-link">Labs</Link><br/>
    </div>
  );
}
```

In the **Kanbas** screen, use the **table** element to layout the **Kanbas Navigation** and the **Dashboard** into a single row with two columns. Put the **Kanbas Navigation** sidebar on the left column, and the routes on the right. Confirm that the **Kanbas** application renders as shown below.

src/Kanbas/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
import Account from "../Account";
import Dashboard from "../Dashboard";
import KanbasNavigation from "../Navigation";
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <table>
        <tr>
          <td valign="top">
            <KanbasNavigation />
          </td>
          <td valign="top">
            <Routes>
              <Route path="/" element={<Navigate to="/Kanbas/Account" />} />
              <Route path="/Account/*" element={<Account />} />
              <Route path="/Dashboard" element={<Dashboard />} />
            </Routes>
          </td>
        </tr>
      </table>
    </div>
  );
}
```

[Northeastern](#)
[Account](#)
[Dashboard](#)
[Courses](#)
[Calendar](#)
[Inbox](#)
[Labs](#)

Dashboard

Published Courses (12)



[CS1234 React JS](#)

Full Stack software developer

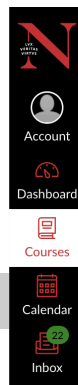
[Go](#)

3.4 Implementing the Courses Screen

Clicking a course in the **Dashboard** navigates to the **Course** screen displaying a course's content as shown here on the right. Create a placeholder for the **Course** screen in file **src/Kanbas/Courses/index.tsx**.

src/Kanbas/Courses/index.tsx

```
export default function Courses() {
  return (
    <div id="wd-courses">
      <h2>Course 1234</h2>
    </div>
  );
}
```



CS5610 SU1 24 MON/FRI > Modules

Home

Modules

Piazza

Zoom Meetings

Assignments

Quizzes

Grades

People

Settings

Collapse All

View Progress

Publish All

+ Module

Week 1, Lecture 1 - Course Introduction, Syllabus, Agenda

LEARNING OBJECTIVES

Introduction to the course

Learn what is Web Development

Add a new route in the **Kanbas** component to navigate to the **Courses** screen when clicking on a course in the **Dashboard Screen**. Optionally add additional placeholder routes as shown. Confirm clicking on a course in the **Dashboard** screen actually navigates to the **Courses** screen.

src/Kanbas/index.tsx

```
import { Routes, Route, Navigate } from "react-router";
import Account from "../Account";
import Dashboard from "../Dashboard";
import KanbasNavigation from "../Navigation";
import Courses from "../Courses";
export default function Kanbas() {
  return (
    <div id="wd-kanbas">
      <table>
        <tr>
          <td valign="top">
            <KanbasNavigation />
          </td>
          <td valign="top">
            <Routes>
              <Route path="/" element={<Navigate to="Account" />} />
              <Route path="/Account/*" element={<Account />} />
              <Route path="/Dashboard" element={<Dashboard />} />
              <Route path="/Courses/:cid/*" element={<Courses />} />
              <Route path="/Calendar" element={<h1>Calendar</h1>} />
              <Route path="/Inbox" element={<h1>Inbox</h1>} />
            </Routes>
          </td>
        </tr>
      </table>
    </div>
  );
}
```

3.4.1 Implement the Course Navigation

When navigating to the **Courses** screen, a second column displays a **Course Navigation** sidebar to navigate to various screens related to that course. In *src/Kanbas/Courses/Navigation.tsx* create the **CoursesNavigation** sidebar as shown below.

src/Kanbas/Courses/Navigation.tsx

```
import { Link } from "react-router-dom";
export default function CoursesNavigation() {
  return (
    <div id="wd-courses-navigation">
      <Link id="wd-course-home-link" to="/Kanbas/Courses/1234/Home">Home</Link><br/>
      <Link id="wd-course-modules-link" to="/Kanbas/Courses/1234/Modules">Modules</Link><br/>
      <Link id="wd-course-piazza-link" to="/Kanbas/Courses/1234/Piazza">Piazza</Link><br/>
      <Link id="wd-course-zoom-link" to="/Kanbas/Courses/1234/Zoom">Zoom</Link><br/>
      <Link id="wd-course-quizzes-link" to="/Kanbas/Courses/1234/Assignments">Assignments</Link><br/>
      <Link id="wd-course-assignments-link" to="/Kanbas/Courses/1234/Quizzes">Quizzes</Link><br/>
      <Link id="wd-course-grades-link" to="/Kanbas/Courses/1234/Grades">Grades</Link><br/>
      <Link id="wd-course-people-link" to="/Kanbas/People">People</Link><br/>
    </div>
  );
}
```

[Home](#)
[Modules](#)
[Piazza](#)
[Zoom](#)
[Assignments](#)
[Quizzes](#)
[People](#)

Then in the **Courses** component, use a **table** to display the sidebar on the left and the **Routes** on the right as shown below. For now, the **Routes** are just placeholders for several screens containing learning materials, assignments, grades, etc. Each screen will be implemented later in the assignment.

src/Kanbas/Courses/index.tsx

```
import CoursesNavigation from "../Navigation";
import { Navigate, Route, Routes } from "react-router";
export default function Courses() {
  return (
    <div id="wd-courses">
      <h2>Course 1234</h2>
      <hr />
      <table>
        <tr>
          <td valign="top">
            <CoursesNavigation />
          </td>
          <td valign="top">
            <Routes>
              <Route path="/" element={<Navigate to="Home" />} />
              <Route path="Home" element={<h2>Home</h2>} />
              <Route path="Modules" element={<h2>Modules</h2>} />
              <Route path="Assignments" element={<h2>Assignments</h2>} />
              <Route path="Assignments/:aid" element={<h2>Assignment Editor</h2>} />
              <Route path="People" element={<h2>People</h2>} />
            </Routes>
          </td>
        </tr>
      </table>
    </div>
  );
}
```

[Northeastern](#)

[Account](#)

[Dashboard](#)

[Courses](#)

[Calendar](#)

[Inbox](#)

[Labs](#)

Course 1234

[Home](#)

[Modules](#)

[Piazza](#)

[Zoom](#)

[Assignments](#)

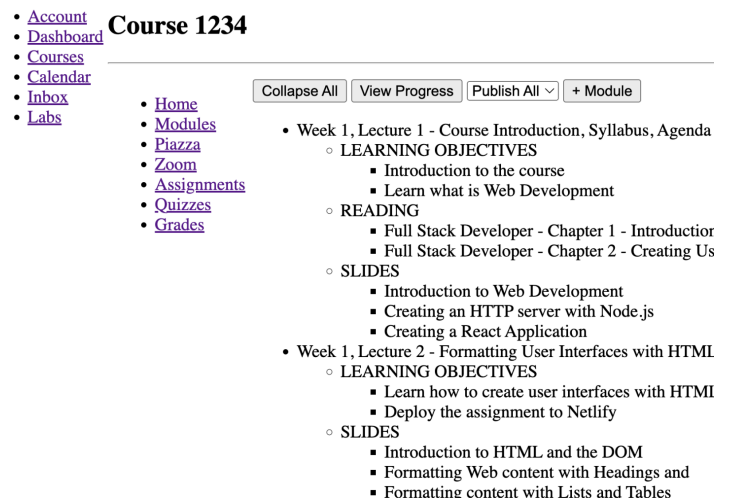
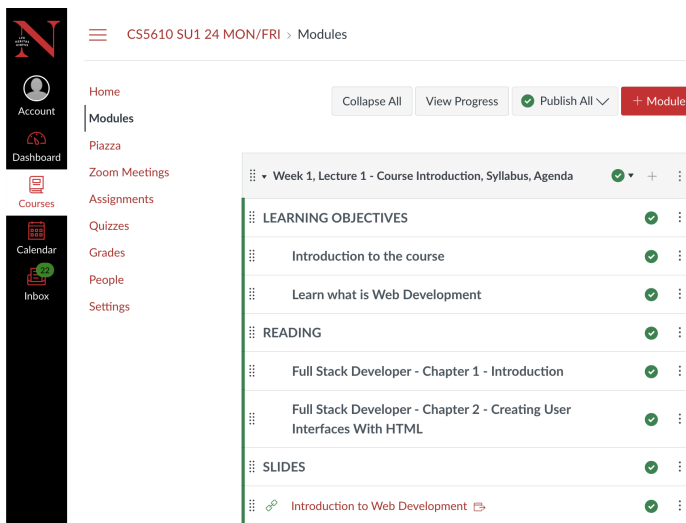
[Quizzes](#)

[People](#)

Home

3.5 Implementing the Modules Screen

When a course is clicked in the **Dashboard**, the application navigates to the **Home** screen of that course by default. The **Home** screen displays a list of modules that contains a course's material. The screen shot below on the left illustrates the first module **Week 1, Lecture 1**, as well as providing links to **READING** material and **SLIDES**. This list of modules is the same content also available in the **Modules** shown below. Implement the **Modules** screen first and then reuse it to implement the **Home** screen. The screen consists of three columns containing the **Kanbas Navigation** sidebar in the first column, a **Course Navigation** sidebar in the second column, and the **Modules** in the third column. The **Kanbas** and **Course Navigation** sidebars are already implemented. Focus now on a prototype of the **Modules** as shown below on the right.



The **Modules** screen can be implemented as a set of nested lists where the top level list consists of a list of **modules**. Each module item can contain a nested list of **lessons**. Each lesson in turn can contain a list of different content items. The code sample below illustrates how nested lists could be used to implement the **Modules**.

src/Kanbas/Courses/Modules/index.tsx

```
export default function Modules() {
  return (
    <div>
      { /* Implement Collapse All button, View Progress button, etc. */ }
      <ul id="wd-modules">
        <li className="wd-module">
          <div className="wd-title">Week 1</div>
          <ul className="wd-lessons">
            <li className="wd-lesson">
              <span className="wd-title">LEARNING OBJECTIVES</span>
              <ul className="wd-content">
                <li className="wd-content-item">Introduction to the course</li>
                <li className="wd-content-item">Learn what is Web Development</li>
              </ul>
            </li>
          </ul>
        </li>
        <li className="wd-module">
          <div className="wd-title">Week 2</div>
          <ul className="wd-lessons">
            <li className="wd-lesson">
              <span className="wd-title">LEARNING OBJECTIVES</span>
            </li>
          </ul>
        </li>
      </ul>
    </div>
  );
}
```

In the **Courses** component, replace the **Modules** header placeholder with the actual **Modules** component implemented above. Confirm that the **Modules** are visible when navigating to a course and then pressing the **Modules** link.

src/Kanbas/Courses/index.tsx

```
import CoursesNavigation from "../Navigation";
import Modules from "../Modules";
import { Navigate, Route, Routes } from "react-router";
export default function Courses() {
  return (
    <div id="wd-courses">
      ...
      <Routes>
        <Route path="/" element={<Navigate to="Home" />} />
        <Route path="Home" element={<h3>Home</h3>} />
        <Route path="Modules" element={<Modules />} />
        <Route path="Assignments" element={<h3>Assignments</h3>} />
        <Route path="Assignments/:aid" element={<h3>Assignment Editor</h3>} />
        <Route path="People" element={<h3>People</h3>} />
      </Routes>
      ...
    </div>
  );
}
```

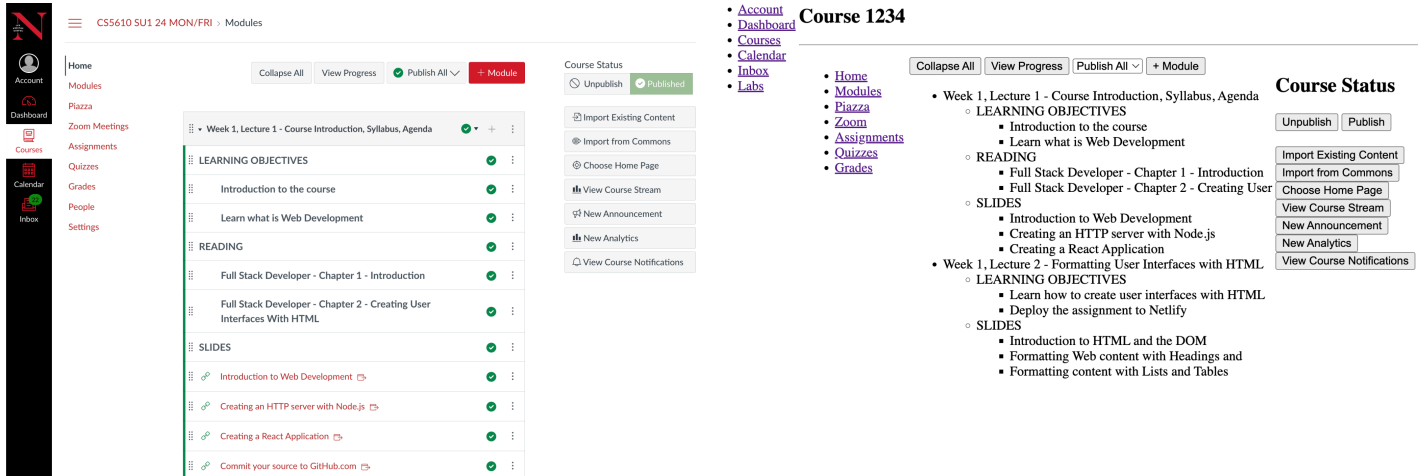
3.6 Implementing the Course Home Screen

Clicking on a course in the **Dashboard** navigates to the course's **Home** screen shown below on the left. The **Home** screen contains four columns containing the **Kanbas** and **Courses Navigation** sidebars in the first and second column, the **Modules** in the third column, and a **Course Status** sidebar on the last column. The content for the first three columns has already been implemented. To implement the **Home** screen, implement the **Course Status** sidebar shown here on the right. Combine the sidebar with the other components to create the **Home** screen as shown below on the right.

Course Status

Unpublish Publish

Import Existing Content
Import from Commons
Choose Home Page
View Course Stream
New Announcement
New Analytics
View Course Notifications



Complete the **Course Status** sidebar on your own in a new file in **src/Kanbas/Courses/Home/Status.tsx**.

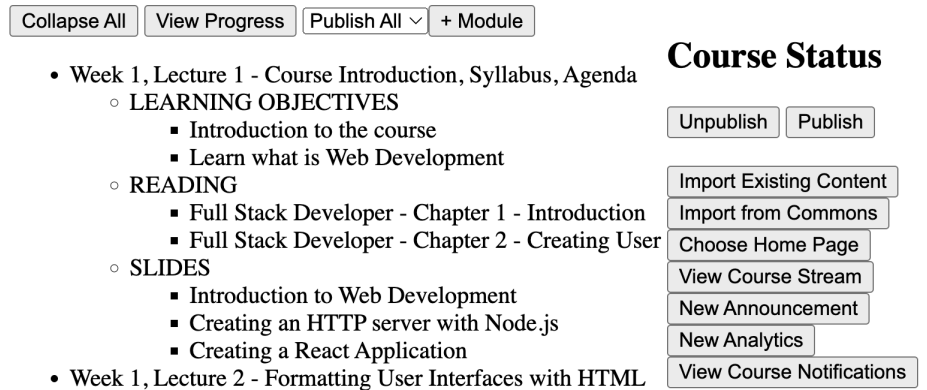
src/Kanbas/Courses/Home/Status.tsx

```
export default function CourseStatus() {
  return (
    <div id="wd-course-status">
      <h2>Course Status</h2>
      <button>Unpublish</button> <button>Publish</button>
      { /* Complete on your own */ }
      <button>View Course Notifications</button>
    </div>
  );
}
```

Combine the **Course Status** with the **Modules** to create the **Home** screen as shown below.

src/Kanbas/Courses/Home/index.tsx

```
import Modules from "../Modules";
import CourseStatus from "../Status";
export default function Home() {
  return (
    <table id="wd-home">
      <tr>
        <td valign="top">
          <Modules />
        </td>
        <td valign="top">
          <CourseStatus />
        </td>
      </tr>
    </table>
  );
}
```



Replace the **Home** heading placeholder in the **Courses** screen with the actual **Home** component as shown below.

src/Kanbas/Courses/index.tsx

```
import CoursesNavigation from "../Navigation";
import Modules from "../Modules";
import Home from "../Home";
import { Navigate, Route, Routes } from "react-router";
export default function Courses() {
  return (
    <div id="wd-courses">
      <h2>Course 1234</h2>
    </div>
  );
}
```

```

<hr />
<table>
  <tr>
    <td valign="top">
      <CoursesNavigation />
    </td>
    <td valign="top">
      <Routes>
        <Route path="/"
          element={<Navigate to="Home" />} />
        <Route path="Home" element={<Home />} />
        <Route path="Modules" element={<Modules />} />
        <Route path="Assignments"
          element={<h3>Assignments</h3>} />
        <Route path="Assignments/:aid"
          element={<h3>Assignment Editor</h3>} />
        <Route path="People" element={<h3>People</h3>} />
      </Routes>
    </td>
  </tr>
</table>
</div>
);}

```

Course 1234

- [Home](#)
- [Modules](#)
- [Piazza](#)
- [Zoom](#)
- [Assignments](#)
- [Quizzes](#)
- [Grades](#)

- Collapse All View Progress Publish All + Module
- Week 1, Lecture 1 - Course Introduction, Syllabus, Agenda
 - LEARNING OBJECTIVES
 - Introduction to the course
 - Learn what is Web Development
 - READING
 - Full Stack Developer - Chapter 1 - Introduction
 - Full Stack Developer - Chapter 2 - Creating User
 - SLIDES
 - Introduction to Web Development
 - Creating an HTTP server with Node.js
 - Creating a React Application
 - Week 1, Lecture 2 - Formatting User Interfaces with HTML
 - LEARNING OBJECTIVES
 - Learn how to create user interfaces with HTML
 - Deploy the assignment to Netlify
 - SLIDES
 - Introduction to HTML and the DOM
 - Formatting Web content with Headings and
 - Formatting content with Lists and Tables

Course Status

Unpublish Publish

Import Existing Content

Import from Commons

Choose Home Page

View Course Stream

New Announcement

New Analytics

View Course Notifications

Confirm that navigating to a course from the dashboard, the **Home** screen displays as shown below.

- [Account](#)
- [Dashboard](#)
- [Courses](#)
- [Calendar](#)
- [Inbox](#)
- [Labs](#)

Course 1234

- [Home](#)
- [Modules](#)
- [Piazza](#)
- [Zoom](#)
- [Assignments](#)
- [Quizzes](#)
- [Grades](#)

Collapse All View Progress Publish All + Module

- Week 1, Lecture 1 - Course Introduction, Syllabus, Agenda
 - LEARNING OBJECTIVES
 - Introduction to the course
 - Learn what is Web Development
 - READING
 - Full Stack Developer - Chapter 1 - Introduction
 - Full Stack Developer - Chapter 2 - Creating User
 - SLIDES
 - Introduction to Web Development
 - Creating an HTTP server with Node.js
 - Creating a React Application
- Week 1, Lecture 2 - Formatting User Interfaces with HTML

Course Status

Unpublish Publish

Import Existing Content

Import from Commons

Choose Home Page

View Course Stream

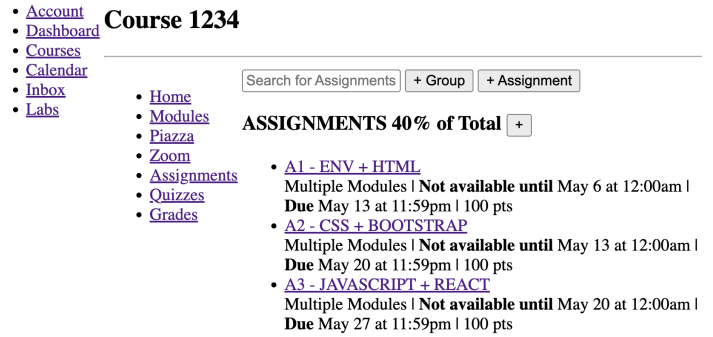
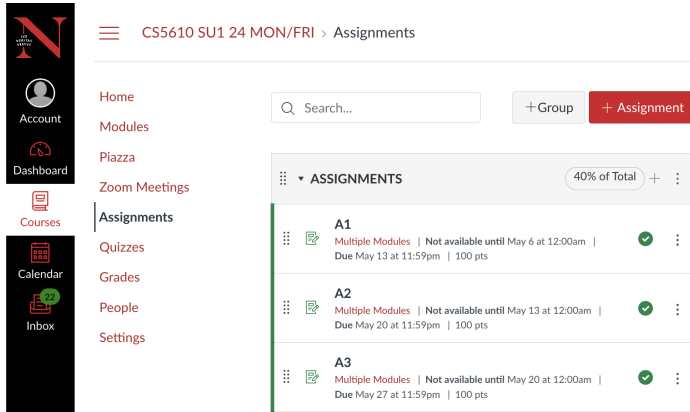
New Announcement

New Analytics

View Course Notifications

3.7 Assignments Screen (On Your Own)

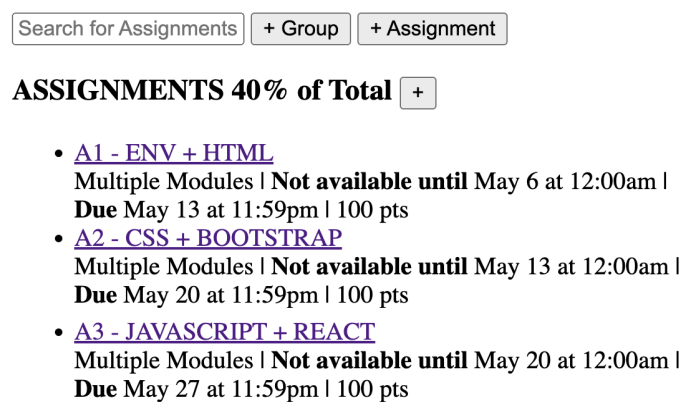
The **Assignments Screen** lists all the assignments students are required to complete throughout a course. A screenshot of the assignments screen is shown below on the left. To navigate to the **Assignments Screen**, users navigate to the course from the **Dashboard**, and then click on the **Assignments** link in the **Course Navigation** sidebar. Assignments are grouped into categories such as **ASSIGNMENTS**, **QUIZZES**, **EXAMS**, and **PROJECT**, and are shown in the third column as shown below. Here only the **ASSIGNMENTS** category is shown and required. Implement a component called **Assignments** shown below on the right as a prototype of the **Assignments** screen on the left.



Use the following starter code to implement the **Assignments** component. Make the component look as shown below on the right. Make sure to keep the **id** and **className** attributes provided and all **li** (line items) in the **ul** (unordered list) must have the **className** set to **wd-assignment-list-item**.

src/Kanbas/Courses/Assignments/index.tsx

```
export default function Assignments() {
  return (
    <div id="wd-assignments">
      <input id="wd-search-assignment"
        placeholder="Search for Assignments" />
      <button id="wd-add-assignment-group">+ Group</button>
      <button id="wd-add-assignment">+ Assignment</button>
      <h3 id="wd-assignments-title">
        ASSIGNMENTS 40% of Total <button>+</button>
      </h3>
      <ul id="wd-assignment-list">
        <li className="wd-assignment-list-item">
          <a className="wd-assignment-link"
            href="#/Kanbas/Courses/1234/Assignments/123">
            A1 - ENV + HTML
          </a>
        </li>
        <li className="wd-assignment-list-item">
          /* Complete On Your Own */
        </li>
      </ul>
    </div>
  );
}
```



In the **Courses** component, add a new **Route** as shown below so that clicking the **Assignments** in the **Courses Navigation** sidebar. Confirm that clicking the **Assignments** link in the **Courses Navigation** actually navigates to the **Assignments** screen.

src/Kanbas/Courses/index.tsx

```
import Assignments from "../Assignments";
...
<Routes>
  <Route path="/" element={<Navigate to="Home" />} />
  <Route path="Home" element={<Home />} />
  <Route path="Modules" element={<Modules />} />
  <Route path="Assignments" element={<Assignments />} />
</Routes>
...
```

3.8 Assignment Editor Screen (On Your Own)

When clicking the title of an assignment in the **Assignments** screen, **Kanbas** navigates to the **Assignment Editor** screen where faculty can edit the assignment's details such as the title, points, and due date as shown below on the left. In **src/Kanbase/Courses/Assignments/Editor.tsx** create the **AssignmentEditor** component that implements the a prototype of the **Assignment Editor** screen as shown below on the right.

Account

Dashboard

Courses

Calendar

Inbox

History

Commons

Studio

Help

CS5610 SU1 24 MO... > Assignments > A1

202440_2 Summer 1 2024 S...

Home

Modules

Piazza

Zoom Meetings

Assignments

Quizzes

Grades

People

Settings

Assignment Name

A1

The assignment is available online

Submit a link to the landing page of your Web application running on [Netlify](#).

The landing page should include the following:

Your full name and section

Links to each of the lab assignments

Link to the Kanbas application

Links to all relevant source code repositories

The Kanbas application should include a link to navigate back to the landing page.

Points

100

Assignment Group

ASSIGNMENTS

Display Grade as

Percentage

Submission Type

Online

Online Entry Options

☐ Text Entry

☒ Website URL

☐ Media Recordings

☐ Student Annotation

☐ File Uploads

Assign to

Everyone

Due

May 13, 2024, 11:59 PM

Available from

May 6, 2024, 12:00 AM

Until

Cancel

Save

Account

Dashboard

Courses

Calendar

Inbox

History

Commons

Studio

Help

Course 1234

Home

Modules

Piazza

Zoom

Assignments

Quizzes

Grades

Assignment Name

A1 - ENV + HTML

The assignment is available online Submit a link to the landing page of your Web application running on Netlify. The landing page should include the following: Your full name and section Links to each of the lab assignments Link to the Kanbas application Links to all relevant source code repositories The Kanbas application should include a link to navigate back to the landing page.

Points

100

Assignment Group

ASSIGNMENTS

Display Grade as

Percentage

Submission Type

Online

Online Entry Options

☐ Text Entry

☐ Website URL

☐ Media Recordings

☐ Student Annotation

☐ File Uploads

Assign Assign to

Everyone

Due

05/13/2024

Available from

05/06/2024

Until

05/20/2024

Cancel

Save

Use the following starter code to get going with the implementation of the **Assignment Editor** screen.

src/Kanbas/Courses/Assignments/Editor.tsx

```
export default function AssignmentEditor() {
  return (
    <div id="wd-assignments-editor">
      <label htmlFor="wd-name">Assignment Name</label>
      <input id="wd-name" value="A1 - ENV + HTML" /><br /><br />
      <textarea id="wd-description">
        The assignment is available online Submit a link to the landing page of
      </textarea>
      <br />
      <table>
```

```
/* clicking this label */
/* selects this field */
```

Copyright © 2024 Jose Annunziato. All rights reserved.

30

<pre> <tr> <td align="right" valign="top"> <label htmlFor="wd-points">Points</label> </td> <td> <input id="wd-points" value={100} /> </td> </tr> </table> { /* Complete on your own */ } </div>);} </pre>	<pre> { /* clicking this label */ } { /* selects this field */ } </pre>
--	--

All form elements (**input** and **select**) must have an **id** attribute set to the following values, based on the labels associated with the element:

wd-points	wd-group	wd-display-grade-as	wd-submission-type	wd-text-entry
wd-website-url	wd-media-recordings	wd-student-annotation	wd-file-upload	wd-assign-to
wd-due-date	wd-available-from	wd-available-until	wd-name	

Clicking the labels above or next to a text fields, gives focus to the field. Clicking the labels next to a checkbox, toggles the checkbox. Clicking the label above a date input field, gives focus to the date input field. In the **Courses** component, add a **Route** to navigate to the new **Assignment Editor** screen when clicking the title of an assignment. For now all assignments navigate to the same **AssignmentEditor** component and display the same information for all assignments. Later chapters will discuss how to retrieve information for a specific assignment and display the corresponding information.

src/Kanbas/Courses/index.tsx

```

import Assignments from "../Assignments";
import AssignmentEditor from "../Assignments/Editor";
...
<Routes>
  <Route path="Home" element={<Home />} />
  <Route path="Modules" element={<Modules />} />
  <Route path="Assignments" element={<Assignments />} />
  <Route path="Assignments/:aid" element={<AssignmentEditor />} />
</Routes>
...

```

4 Source Control

4.1 Install a git client

On macOS you already get a command line git client. You can fully interact with **github.com** from the terminal. On Windows OS, you'll need to install a git client from where you will be able to issue the same commands from a console. Download git for windows from <https://git-scm.com/download/win>, run the installer and follow the instructions. At the end of the installation you should be able to execute git commands from new console instances. All examples in this course assumes you have git installed in your OS.

You can also install a graphical git client if you prefer. There are a lot of alternatives, but the author prefers Sourcetree since it works well and consistently in both macOS and Windows. To install Sourcetree download from <https://www.sourcetreeapp.com/>, install, and follow instructions. We will not be covering how to use Sourcetree, but you are free to use it if you wish. All examples in this course will be using the command line git client.

4.2 Ignoring files and directories

Git can keep track of all the files in your project, but there are some files or directories that you don't want to keep track of, for instance, compiled classes, libraries, etc. You can configure which files and directories you want git to ignore by listing them in a file called **.gitignore** at the root of your project, which should already exist. Create the file if it does not already exist. With a text editor or from IntelliJ, edit the file **.gitignore**. Towards the top of the file, in a blank line, type the following. **Note** the period at the beginning of the file!!

```
.gitignore
.idea
node_modules
```

This tells git that the **.idea** folder should be ignored because it is a directory specific to IntelliJ and not relevant to React.js project itself. If you are using other IDEs, then you might want to add other files or directories here relevant to your specific IDE. The **node_modules** folder should also be ignored since it contains library files that should not be added to source control. **Note:** make sure that IDE specific folders and the **node_modules** folders don't make it into the repository!!

4.3 Create a remote source repository

If you don't already have an account on **github.com**, create a new account or use an account you already have at **github.com**. Do not use the university's github or your work's source control if you already have one. Login to your **github.com** account and click the **New** button to create a new **public repository** called **kanbas-react-web-app**, just like the name of the React.js project you created earlier. Here's an example on how it looks on my laptop. Your username "**jannunzi**" will obviously be different.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk ().*

Owner *

 jannunzi ▾

Repository name *

/ kanbas-react-web-app

✔ kanbas-react-web-app is available.

Once you create the remote repository on github, it will display commands on how to commit and push your code from your computer up to the remote repository. The commands will be similar to the ones shown below. The username "**jannunzi**" will obviously be different.

Quick setup — if you've done this kind of thing before



Set up in Desktop

or

HTTPS

SSH

<https://github.com/jannunzi/kanbas-react-web-app.git>



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# kanbas-react-web-app" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/jannunzi/kanbas-react-web-app.git
git push -u origin main
```



4.4 Adding and committing your code

Now that we have a remote source repository, let's add, commit and push our code to the repository. From the terminal or console, make sure you are in the **kanbas-react-web-app** directory and then type the following commands which are based on the commands git suggested. Ignore the commentary on the right. Also, your actual URL below will differ for you, so **don't blindly copy the example below**. Use the commands git suggested for you.

```
$ git init                                # initializes local repository
$ git add .                               # adds all files to repository
$ git commit -m "first commit"            # commits files with message
$ git remote add origin https://github.com/jannunzi/kanbas-react-web-app.git # adds remote repository
$ git push -u origin main                 # copies local files to remote
```

Refresh the remote repository on github.com and confirm that the files are now available there

4.5 Using personal access tokens

While pushing to GitHub you might encounter an error stating that **password authentication was removed on August 13, 2021**. One way to fix this is to generate a **personal access token** and use that instead of your password. To generate a personal access token go to your GitHub **Settings**, and then **Developer Settings**. Click on **Personal access tokens** and then on **Generate new token**. Enter a short description in the **Note** field, select **No expiration** for **Expiration**, and grant all access privileges by selecting all the checkboxes under **Select scopes**. You are welcome to be more restrictive if you want. Click on **Generate token** and copy the long unique access token to your clipboard. Note that this will be the only opportunity to copy the token and you fail to do so you'll have to delete this token, create a brand new one, and try again. With the token copied to the clipboard, try pushing again to GitHub, but this time paste the token when asked for a password.

If you are not being asked for a password then GitHub might be using a cached authentication. To clear cached GitHub authentications on Windows go to the **Credential Manager**, click on **Windows Credentials**, click the GitHub credentials, and click **Remove**. This time github should ask for your username and password again when trying to push. Paste the access token when asked for the password. To clear cached authentications on macOS, go to your **Key Chain** and search for github. Remove the GitHub key chain.

5 Deploying to a remote server

Create an account at <https://www.netlify.com/> if you don't already have one. Follow the instructions to create a team or organization and then navigate to the **Team overview** screen. On the **Team overview** or **Sites** page, click on **Create/Add new site**. Select "Import an existing project".

- Connect to **Git provider**, select **Github**. Give **Netlify** all the authorizations it asks for. **Pick a repository**, in the **Search repos** field. **Lookup and pick the kanbas-react-web-app** repository you created earlier
- In the **Site settings, and deploy** tab, select the branch to deploy from, e.g., **master or main**, and click **Deploy site**. The deployment process might take a few minutes. Netlify will pick a random silly name for your application, e.g., **loving-torvalds-effde8**. That's fine for now, we can setup custom domain names later. While your project deploys it will show the main steps it's going through and will eventually say **Published** in green half way down the screen. **You can then click on the URL towards the top, e.g., <https://loving-torvalds-effde8.netlify.app> and see your project deployed.**

In the **Deployment settings**, disable **Continuous Integration** so your deployment is not rejected by trivial warnings. From your Netlify dashboard, click on **Deployment settings**, then **Build & deploy**, and then **Continuous deployment**. Scroll down to **Build configuration** and click on **Configure**. Change the **Build command** from `npm run build` to `CI=false npm run build`.

Also enable all branches to deploy in separate URLs so that TAs can grade each assignment individually while allowing you to continue working on the next assignment. Scroll further down to **Branches and deploy contexts** and click **Configure**. In the section **Branch deploys** select the radio button **All**. Now all the branches pushed to the repository will be available in separate URLs.

6 Deliverables

1. Download and install the latest **Node.js** as described.
2. Create a React.js application called **kanbas-react-web-app** as described.
3. Push the source code of the React.js application **kanbas-react-web-app** to a remote source repository in **GitHub.com** as described
4. **Labs/index.tsx** contains a **TOC.tsx** that references each of the labs and Kanbas. Add a link to your repository in GitHub. The link should have an **ID** attribute with a value of **wd-github**.
5. In **Labs/index.tsx**, add your full name: first name first and last name second. Use the same name as in Canvas.
6. Deploy the **kanbas-react-web-app** React.js application to **Netlify** as described.
7. As a deliverable in **Canvas**, submit the URL to your React.js application running on Netlify.

In the **Kanbas** component, update the **Account** route to render the **Account** component. When users click the **Account** link in the **Kanbas Navigator**, they should see the **Sign in** screen.

Add the **Account** component to the **Kanbas** router as shown below. Confirm that navigating to the **Account** screen displays the **Sign in** screen by default.

src/Kanbas/index.tsx

```
<Routes>
  <Route path="/Account/*" element={}<Account /> } />
  <Route path="/Dashboard" element={}<Dashboard /> } />
  <Route path="/Courses/:cid/*" element={}<Courses /> } />
</Routes>
```